

Assign 10:

For this assignment, I used concepts of file io, array sorting, loops, conditionals, recursion and vectors/To read the data from the text file I used file IO to store as they allow dynamics resizing and are stored in heap in contrast to stack. I only sorted the words file.

To sort the data I used quick Algorithm A and selection sort for Algorithm(B). By sorting the data set in an ascending order from low to high (A-Z)

Algorithm A (Quick sort): Is highly efficient as it uses divide and conquering. Where It works by selecting a pivot element from the array or vector and then partitioning the other elements into two smaller sub-arrays: those elements less than the pivot and those greater than the pivot
Selecting a pivot element from the array (in this implementation, the last element)**Partitioning** the array so that: All elements smaller than or equal to the pivot are moved to its left. All elements greater than the pivot are moved to its right. Then Recursively applying** the same process to the left and right sub-arrays. The algorithm has an average time complexity of: $O(n \log n)$ and a worst-case time complexity of $O(n^2)$.

Algorithm B (Selection Sort): Works on finding the smallest element in the unsorted and swapping it with the first unsorted element using an iterative process of $(n-1)$ rounds where n is there number of elements in the array/vector the iterations stop when the data is sorted. This algorithm has an average time and worst-case time complexity $O(n^2)$. Selection Sort is inefficient for large datasets but has the advantage of simplicity and minimal memory usage.

Results from compilation

After sorting the 5000-word dataset, the execution times were:

- **Quick Sort:** 4,695 microseconds
- **Selection Sort:** 459,134 microseconds

These results clearly demonstrate that Quick Sort is significantly faster and more scalable than Selection Sort for large datasets.